

Quantum Variational Algorithms on Adiabatic Quantum Computing Devices

Philipp Hanussek

June 16, 2026

1 Motivation

Since the development of the quantum computers, many claims regarding quantum advantage have been made. Often, these advantage claims did not last long. In this work, we focus exclusively on the D-Wave Quantum Annealer, which leverages quantum technology to solve quadratically unconstrained binary optimization (QUBO) problems. D-Wave is among the first companies to commercially sell quantum computers. In 2025, researcher from D-Wave claimed to have solved a scientifically relevant problem fast using quantum technology, than it would have been possible on classical devices. However, it is currently an open question whether the D-Wave Quantum Annealer is able to solve other problems well, and possible faster than conventional computers. In this work, we assess the D-Wave's ability to sample from a probability distribution, to approximate the ground state of different statistical mechanical models using variational algorithms.

1.1 Previous Work

The D-Wave's Quantum Annealers performance scaling for finding the ground state (corresponding to the minimum of the quadratic binary cost function) has been extensively studied, with classical methods exhibiting better scaling and solution times. This has been done on a wide range of problems, ranging from cryptographic instances to physics inspired models.

2 Sampling Methods

In this section we briefly outline the main computational and algorithmic methods used in this work.

2.1 Quantum (inspired) solvers

Here we present two solver types: First, a special quantum machine designed to find the ground state of an Ising Hamiltonian, and second, a classical quantum inspired algorithm.

2.1.1 Adiabatic Quantum Computing

Adiabatic Quantum Computing is based on the adiabatic theorem, which states: *A physical system remains in its instantaneous eigenstate if a given perturbation is acting on it slowly enough and if there is a gap between the eigenvalue and the rest of the Hamiltonian's spectrum.* In this work we focus on the D-Wave Quantum Annealer, which acts according to the adiabatic theorem in the following way: In the beginning of the annealing process, the device is initiated in the ground state of a initial driver Hamiltonian H_D that can be easily constructed:

$$H_D = - \sum_{i=1}^N \sigma_x^{(i)} \text{ with ground state } |+\rangle^{\otimes N} \quad (1)$$

where σ_x , σ_y , and σ_z are the Pauli matrices acting on qubit i . The problem Hamiltonian is defined as

$$H_P = \sum_{i=1}^N h_i \sigma_z^{(i)} + \sum_{i,j=1}^N J_{ij} \sigma_z^i \otimes \sigma_z^j. \quad (2)$$

During the annealing process, the system is slowly brought from H_D to H_P according to the so-called annealing schedules $A(s)$ and $B(s)$:

$$H(s) = A(s)H_D + B(s)H_P = -A(s) \sum_{i=1}^N \sigma_x^{(i)} + B(s) \sum_{i=1}^N h_i \sigma_z^{(i)} + B(s) \sum_{i=1}^N \sum_{j=1}^{i-1} J_{ij} \sigma_z^{(i)} \otimes \sigma_z^{(j)}, \quad (3)$$

where $s \in [0, 1]$ is the normalized time. $A(s)$ decreases the driving terms in H_D as s increases, and $B(s)$ activates the terms of the problem Hamiltonian H_P . At the end of the evolution, if the process happens slow enough, and assuming a noiseless environment, the system will be in the ground state of the problem Hamiltonian according to the adiabatic theorem.

2.1.2 Simulated Bifurcation

Simulated Bifurcation (SB) is a heuristic algorithm that numerically simulates adiabatic and ergodic (chaotic) evolutions of classical nonlinear Hamiltonian systems to solve combinatorial optimization problems. Inspired by quantum adiabatic optimization using networks of Kerr-nonlinear parametric oscillators (KPO), the algorithm encodes the ground state of an Ising Hamiltonian into the stable states of a network of classical oscillators.

Each Ising spin is represented by continuous dynamical variables, position x_i and momentum y_i , which serve as canonical conjugate variables for each oscillator. The system evolves according to coupled differential equations derived from a network of Duffing oscillators. These equations include a nonlinear x^4 term and a time-dependent pumping parameter $p(t)$. As $p(t)$ is gradually increased, the system undergoes an adiabatic bifurcation, in which a single stable equilibrium at the origin splits into two stable branches. This mechanism drives the trajectories toward configurations corresponding to low-energy states of the Ising Hamiltonian. Because all variables are updated simultaneously at each time step, the algorithm is highly parallelizable and well suited for implementation on FPGAs and GPUs.

Langevin Simulated Bifurcation Langevin Simulated Bifurcation (LSB) is a stochastic adaptation of SB designed to function as a Boltzmann sampler for training energy-based models such as Semi-Restricted Boltzmann Machines (SRBMs). While SB is a deterministic optimization method, LSB introduces stochasticity to approximate a Boltzmann distribution $B_{\beta_{\text{eff}}}$ with an unknown effective inverse temperature β_{eff} .

LSB preserves the parallel symplectic Euler update rules of SB but introduces a stochastic reset at the end of each iteration: the position variables are discretized $x_i \rightarrow \text{sgn}(x_i)$, and the momentum variables are re-initialized according to a Gaussian distribution $y_i \sim \mathcal{N}(0, \sigma)$. This prevents convergence to a single minimum and allows exploration of the energy landscape, enabling sampling performance comparable to sequential MCMC methods at significantly higher computational speed. To handle the unknown β_{eff} during learning, LSB is often combined with Conditional Expectation Matching (CEM).

Conditional Expectation Matching CEM accounts for the fact that, in order to produce samples using LSB, a parameter β_{eff} , which is unknown and instance dependent, has to be found. This parameter is called the effective inverse temperature and it defines the distribution returned by the sampler. In Semi-Restricted Boltzmann Machine (SRBM), hidden variables are conditionally independent when the visible units are fixed to a condition vector r . This allows the direct calculation of the expectation value of a hidden unit:

$$\langle h \rangle_{\beta, r} \quad (4)$$

2.2 Metropolis-Hastings inspired methods

We use several Markov chain Monte Carlo (MCMC) methods to generate samples from probability distribution from which direct sampling would be difficult. All of these methods can be traced back to the Metropolis-Hastings algorithm, which we explain below.

2.2.1 Metropolis-Hastings Algorithm

The Metropolis-Hastings Algorithm is a MCMC method used to obtain samples from a (multivariate) probability distribution $P(x)$ from which sampling is difficult. The goal of the algorithm is to define a Markov chain whose stationary distribution is $P(x)$. Suppose we can evaluate an unnormalized target density $f(x)$, where $f(x) \propto P(x)$, and the normalization constant is unknown. In the **proposal** step, a *candidate* x^* from a proposal distribution $q(x^*|x_n)$ is generated, x_n being the current state of the Markov chain. In the **accept-reject** step, the acceptance probability $A(x_n \rightarrow x^*)$ is calculated[1]:

$$A(x_n \rightarrow x^*) = \min \left(1, \frac{f(x^*)}{f(x_n)} \times \frac{q(x_n|x^*)}{q(x^*|x_n)} \right). \quad (5)$$

For the evaluation of $\frac{f(x^*)}{f(x^n)}$, knowledge of the normalization constant is not necessary. The candidate x^* is accepted with probability $A(x_n \rightarrow x^*)$.

2.2.2 Simulated Annealing

Simulated Annealing is the classical analogue and predecessor of quantum annealing. It works by exploring the neighborhood of a given configuration, accepting the proposed solution with a temperature dependent probability, or if the new configuration has a lower energy than the current one.

2.2.3 Gibbs sampling

Gibbs sampling can be considered a special case of the Metropolis-Hastings algorithm. It is used when sampling directly from a joint distribution is difficult, but sampling from the full conditional distributions is tractable. The algorithm constructs a Markov chain by iteratively sampling each variable (or block of variables) from its distribution conditioned on the current values of all other variables. This procedure generates a chain whose stationary distribution is the target joint distribution.

3 Variational Algorithms

Variational algorithms are a class of hybrid quantum-classical optimization methods. We focus on the Variational Quantum Eigensolver (VQE) which is designed to find the ground state of a given Hamiltonian. It consists of three parts: the *cost function* to be minimized, the *ansatz*, which in our case is a Boltzman machine, and an *expectation value estimation*. Additionally, a classical optimizer is needed to improve the ansatz at each iteration. Depending on the quantum architecture, two approaches for estimating the expectation value are possible:

- On gate-based quantum computers, measuring can be used to gather expectation values of the modified ansatz and improve it further
- In adiabatic quantum computing, the quantum device is used to generate representative samples of the modified ansatz. These samples are then used to approximate the expectation value.

Mathematically, the problem can be described as follows: The goal is to approximate the ground state, that is, the eigenstate corresponding to the lowest eigenvalue of a Hamiltonian H . In physics, a Hamiltonian is defined to be an operator that describes the energy of a system. Efficient methods exist to encode many classical problems, such as scheduling or integer factoring, into Hamiltonians. The ground state of this Hamiltonian is often used to represent the problem's solution. Calculating this solution analytically using for example exact diagonalisation is computationally difficult, as the Hilbert space grows exponentially with the size of the quantum state. In the adiabatic case, the ansatz can be written as

$$|\Phi\rangle = \sum_v \Phi(v) |v\rangle, \text{ with } \Phi(v) = \sum e^{-\Phi(v,h)} \quad (6)$$

The network is specified by the weights $\mathbf{a}, \mathbf{b}, \mathbf{W}$, which are respectively applied to the vector of visible units $\mathbf{v} = (v_1, \dots, v_N)$ and hidden units $\mathbf{h} = (h_1, \dots, h_N)$.

References

- [1] Danielle Navarro. The Metropolis-Hastings algorithm. https://blog.djnavarro.net/posts/2023-04-12_metropolis-hastings/#fnref1. [Accessed 16-06-2026].